

Sasha Murray and Ved Barakam

Midterm 1 Review

Polars, SQL, Data Cleaning, EDA, Regex, Visualizations

Slides adapted from James Geronimo, Sara Eginova, Sarika Pasumarthu, and Kelly Hu (2025-2026)

Fall 2026

Polars

Polars

Polars is a fast Python library for processing tabular data, written in Rust.

- Process data in tables (DataFrames)
- Multi-threaded, vectorized operations (very fast)

Built around **expressions**



Importing Polars:

```
import polars as pl
```

DataFrame

DataFrames represent tabular data. Each column is a named Series with a single data type (shown under the column name).

shape: (3, 3)

fruit --- str	stock --- i64	price --- f64
apple	2	1.99
banana	3	0.99
mango	5	5.99

`df.shape` → (3, 3)

`df.columns` → list of column names

`df.schema` → column names + dtypes

`df["price"]` → the price Series

Expressions

Most Polars code is built from **expressions** : a description of a computation on a column, written with `pl.col()`.

```
(pl.col("stock") * pl.col("price")).round(2).alias("value")
```

DataFrame Slicing

```
df.head(n) / df.tail(n)
```

first / last n rows

```
df.select("fruit", "price")
```

columns by name (list works too)

```
df.filter(pl.col("stock") > 2)
```

rows by condition

```
df[0:2, ["fruit", "price"]]
```

position-based rows + columns (what type does this return?)

```
df.item(0, "price")
```

single value → 1.99 (what type does this return?)

```
df.row(0)
```

one row as a tuple

DataFrame Operations - Filtering

Keeping rows that match a condition:

```
df.filter((pl.col("price") > 1) & (pl.col("stock") < 5))
```

Combine with & (and), | (or), ~ (not) — wrap each in parentheses

`pl.col("fruit").is_in(["apple", "mango"])` for membership

shape: (1, 3)

fruit	stock	price
---	---	---
str	i64	f64
apple	2	1.99

DataFrame Operations - New Column

Creating new columns:

```
df = df.with_columns(  
    (pl.col("stock") * pl.col("price")).alias("value"),  
)
```

shape: (3, 5)

fruit --- str	stock --- i64	price --- f64	yumminess --- i32	value --- f64
apple	2	1.99	100	3.98
banana	3	0.99	100	2.97
mango	5	5.99	100	29.95

DataFrame Operations - Rename Column(s)

Renaming columns:

```
df = df.rename({"stock": "quantity", "price": "cost"})
```

- Maps old column names → new column names

Inside an expression, rename with `.alias("new_name")`

DataFrame Operations - Drop Column(s)

Removing columns:

```
df = df.drop("yumminess", "value")
```

To remove rows, use `df.filter(...)` with the opposite condition

Can remove rows with missing values: `df.drop_nulls()`

DataFrame Operations - Sorting

Sorting by column(s):

```
df = df.sort("price", descending=True)
df = df.sort(["fruit", "price"], descending=[False, True])
```

ascending order is the default

- Can sort by multiple columns

DataFrame Operations - Grouping

Group rows that share a value, then aggregate each group inside `.agg()`:

```
df.groupby("fruit").agg(pl.col("sales").sum())
```

1. Start with df

fruit	sale
apple	3
banana	2
apple	5
mango	4
banana	1
apple	2

`group_by("fruit")`



2. Split into groups

apple	3
apple	5
apple	2
banana	2
banana	1
mango	4

`.agg(...sum())`



3. One row per group

fruit	sale
apple	10
banana	3
mango	4

Other aggregations work the same way: `.mean()`, `.min()`, `.max()`, `.count()`. Pass `maintain_order=True` to keep groups in first-seen order.

DataFrame Operations - Joining

Join two DataFrames on a shared column:

```
left_df.join(right_df, left_on="fruit", right_on="name", how="inner")  
left_df.join(right_df, on="fruit")    # same name in both
```

how : "inner", "left", "right" , "full", "cross"

Helpful Functions - value_counts()

Counts how many times each unique value appears in a column.

```
df["fruit"].value_counts(sort=True)
```

Returns a **DataFrame**

Columns: the original name + count

Unsorted by default — pass sort=True

shape: (3, 2)

fruit --- str	count --- u32
apple	3
banana	2
mango	1

Helpful Functions - Inspecting Data

```
df.describe()
```

DataFrame of summary stats (count, null_count, mean, std, min, quartiles, max)

```
df.glimpse()
```

prints each column sideways with its dtype and first values

```
df.null_count()
```

1-row DataFrame: number of nulls per column

```
s.unique()
```

Series of distinct values (maintain_order=True keeps first-seen order)

```
s.n_unique()
```

int: number of distinct values

```
df.unique(subset="fruit")
```

DataFrame with duplicate rows removed (like drop_duplicates)

Helpful Functions - when / then / otherwise

Building a column from conditions:

```
df.with_columns(  
    pl.when(pl.col("price") > 5)  
      .then(pl.lit("pricey"))  
      .when(pl.col("price") > 1)  
      .then(pl.lit("normal"))  
      .otherwise(pl.lit("cheap"))  
      .alias("tier")  
)
```

Conditions are checked in order; the first match wins.

fruit	price	tier
---	---	---
str	f64	str
apple	1.99	normal
banana	0.99	cheap
mango	5.99	pricey

Helpful Functions - Expr.str (Cleaning Text)

String methods live under `.str` on a column expression. Each one runs on every value in the column, and they can be chained.

```
df.with_columns(  
    pl.col("fruit")  
        .str.strip_chars()      # trim spaces  
        .str.to_lowercase()  
        .str.replace("_", " ")  
        .alias("clean")  
)
```

fruit		clean
" Green_Apple "	→	"green apple"
"BANANA"	→	"banana"
" Root_Beer"	→	"root beer"
"Kiwi "	→	"kiwi"

Helpful Functions - Expr.str (Common Methods)

What each method returns for the value "red apple":

<code>.str.to_uppercase()</code>	"RED APPLE"
<code>.str.len_chars()</code>	9
<code>.str.slice(0, 3)</code>	"red"
<code>.str.replace_all("p", "b")</code>	"red abble"
<code>.str.contains("apple")</code>	true
<code>.str.starts_with("red")</code>	true
<code>.str.split(" ")</code>	["red", "apple"]

Boolean results plug straight into a filter: `df.filter(pl.col("fruit").str.contains("apple"))`

Polars Practice

Fall 2025 Midterm 1

1) Polars? More like Polar Bears [7.5 Pts]

Click [here](#) for the reference sheet. If you need scratch paper, please request some from a proctor.

shape: (3, 2)

a	b
<i>i64</i>	<i>i64</i>
5	8
3	7
6	4

The DataFrame above is called `df`, and `polars` has been imported as `pl`. Consider the following code snippets. If the code snippet will return an integer between 0 and 8, **write the integer that is returned**. If the code snippet will return something other than an integer between 0 and 8, or will return an error, **write NA**.

Note: This question has been translated to Polars from Pandas. Keep in mind that there could be translation errors. Let us know if you spot any!

(i) [1.5 Pt] `df['b'][1]`

i

?

(ii) [1.5 Pt] `df['b', :][1]`

ii

?

(iii) [1.5 Pt] `df[1, 2]`

iii

?

(i) [1.5 Pt] `df['b'][1]`

i 7



(ii) [1.5 Pt] `df['b', :][1]`

ii NA



(iii) [1.5 Pt] `df[1, 2]`

iii NA



(iv) [1.5 Pt] `df.filter(pl.col('b') == 4)['a'][0]`

iv

?

(v) [1.5 Pt] `df.sort('a', descending=True).head(1)['b'][0]`

v

?

(iv) [1.5 Pt] `df.filter(pl.col('b') == 4)['a'][0]`

iv 6

(v) [1.5 Pt] `df.sort('a', descending=True).head(1)['b'][0]`

v 4

Spring 2025 Midterm 1

1 Awesome DATA 100 Staff [19 Pts]

At the end of the semester, Data 100 instructors analyze staff performance using a `DataFrame` called `performance`. The columns of `performance` are described below:

- `name`: Name of the staff member (`type = str`).
Note: No two staff members have exactly the same name.
- `role`: One of three possible staff roles: "GSI", "TA", or "Tutor" (`type = str`).
- `is_graduating`: True if the staff member is graduating this semester and False otherwise (`type = bool`).
- `disc_day`: The staff member's discussion day. One of three values: "W" (Wednesday), "Th" (Thursday), or NaN if the staff member does not hold any discussions (`type = str`).
- `oh_tickets`: Total office hours (OH) tickets the staff resolved during the semester (`type = np.int64`).
- `ed_hours`: Total hours the staff spent resolving questions on EdStem during the semester (`type = np.float64`).
- `grad_day`: The staff member's final day as an enrolled UC Berkeley student, in "yyyy-mm-dd" format (`type = str`).

The first five rows of `performance` are shown below:

	name	role	is_graduating	disc_day	oh_tickets	ed_hours	grad_day
0	Dan	TA	False	W	111	11.9	2026-05-16
1	Gisella	Tutor	True	NaN	117	13.1	2025-05-17
2	Steven	Tutor	False	NaN	109	14.6	2026-08-15
3	Malavikha	TA	True	NaN	122	19.0	2025-05-17
4	Rose	GSI	True	W	4	23.5	2025-05-17

- (h) Instructors want to assess office hours and EdStem performance by staff role.

Assign `staff_stats` to a DataFrame with one row for each unique `role`. For each role, `staff_stats` should contain the **minimum** number of `oh_tickets` and the **maximum** number of `ed_hours` among staff members in that role. Fill in the blanks to achieve this.

Note: Polars DataFrames do not have an index, so `role` should remain as a column in the resulting DataFrame. The resulting DataFrame should contain exactly three columns: `role`, `oh_tickets`, and `ed_hours`. The order of the columns does not matter.

```
staff_stats = performance.group_by(_____(i)_____.agg(  
    _____(ii)_____,  
    _____(iii)_____  
)
```

- (i) [0.5 Pts] Fill in blank (i):

- (ii) [0.5 Pts] Fill in blank (ii):

- (iii) [0.5 Pts] Fill in blank (iii):

(h) Instructors want to assess office hours and EdStem performance by staff role.

Assign `staff_stats` to a DataFrame with one row for each unique `role`. For each role, `staff_stats` should contain the **minimum** number of `oh_tickets` and the **maximum** number of `ed_hours` among staff members in that role. Fill in the blanks to achieve this.

Note: Polars DataFrames do not have an index, so `role` should remain as a column in the resulting DataFrame. The resulting DataFrame should contain exactly three columns: `role`, `oh_tickets`, and `ed_hours`. The order of the columns does not matter.

```
staff_stats = performance.group_by("role").agg(  
    pl.col("oh_tickets").min(),  
    pl.col("ed_hours").max()  
)
```

- (i) [1.5 Pts] Instructors want to recognize top performers on EdStem who **do not hold discussion sections**. A top-performer is any staff member who spent **at least 15 hours** on EdStem.

Note: For staff without discussion sections, `disc_day` is null.

Fill in the following blank to assign `top_ed` to a modified version of the `performance` DataFrame that includes all rows corresponding to these staff members:

```
top_ed = performance.filter(  
    _____(A)_____  
)
```

Fill in blank (A) below.

Answer:

- (i) [1.5 Pts] Instructors want to recognize top performers on EdStem who **do not hold discussion sections**. A top-performer is any staff member who spent **at least 15 hours** on EdStem.

Note: For staff without discussion sections, `disc_day` is null.

Fill in the following blank to assign `top_ed` to a modified version of the `performance` DataFrame that includes all rows corresponding to these staff members:

```
top_ed = performance.filter(  
    (pl.col("ed_hours") >= 15) & pl.col("disc_day").is_null()  
)
```

- (l) Finally, the instructors want to remove all rows where any value is missing and retain only those corresponding to staff members graduating in 2026.

Follow the steps and fill in the blanks to achieve this:

Step 1: Remove all rows with missing (null) values.

```
perf_no_missing = performance._____(i)_____()
```

```
# Step 2: Extract the year from the "grad_day" column,  
#         convert it to an integer, and store it in a  
#         new column called "year".
```

```
perf_no_missing = perf_no_missing.with_columns(  
    _____(ii)_____  
)
```

```
# Step 3: Filter the DataFrame to keep staff members  
#         who are graduating in 2026.
```

```
filtered_perf = perf_no_missing.filter(  
    _____(iii)_____  
)
```

- (i) [0.5 Pts] Fill in blank (i):

- (ii) [1 Pt] Fill in blank (ii):

Note: For Step 2, you **must use string slicing** to complete the task. You may **not** convert `grad_day` to a date or use the `.dt` namespace.

- (iii) [1 Pt] Fill in blank (iii):

- (1) Finally, the instructors want to remove all rows where any value is missing and retain only those corresponding to staff members graduating in 2026.

Follow the steps and fill in the blanks to achieve this:

```
# Step 1: Remove all rows with missing (null) values.
```

```
perf_no_missing = performance.drop_nulls()
```

```
# Step 2: Extract the year from the "grad_day" column,
```

```
#         convert it to an integer, and store it in a
```

```
#         new column called "year".
```

```
perf_no_missing = perf_no_missing.with_columns(  
    pl.col("grad_day").str.slice(0, 4).cast(pl.Int64).alias("year")  
)
```

```
# Step 3: Filter the DataFrame to keep staff members
```

```
#         who are graduating in 2026.
```

```
filtered_perf = perf_no_missing.filter(  
    pl.col("year") == 2026  
)
```

SQL

Tables and Schema

Column or Attribute or Field

Row or
Record or
Tuple

name TEXT, PK	year INT, >=2000	cute INT
hiccup	2010	10
drogon	2011	-100
dragon 2	2019	0

Dragon

Relation (table) name

SQL Syntax

```
SELECT <column(s)>,  
    CASE WHEN condition THEN value  
        [WHEN condition THEN value]  
    ELSE value  
    END AS <column>  
  
FROM <table1> AS a JOIN <table2> AS b ON condition  
  
WHERE <condition> AND <condition>  
  
GROUP BY <column(s)> HAVING <condition>  
  
ORDER BY <column> DESC  
  
LIMIT <n>  
  
OFFSET <number of rows>;
```

Note: Single quotes for 'strings' | Double/No quotes for "table and column names"

GROUP BY

Equivalent to polars `.group_by()`, used to aggregate across the rows of our dataset.

- **Common aggregation functions:** COUNT, MIN, MAX, SUM, AVG

```
%%sql
SELECT *
FROM Dish;
```

name	type	cost
ravioli	entree	10
ramen	entree	13
taco	entree	7
edamame	appetizer	4
fries	appetizer	4
potsticker	appetizer	4
ice cream	dessert	5

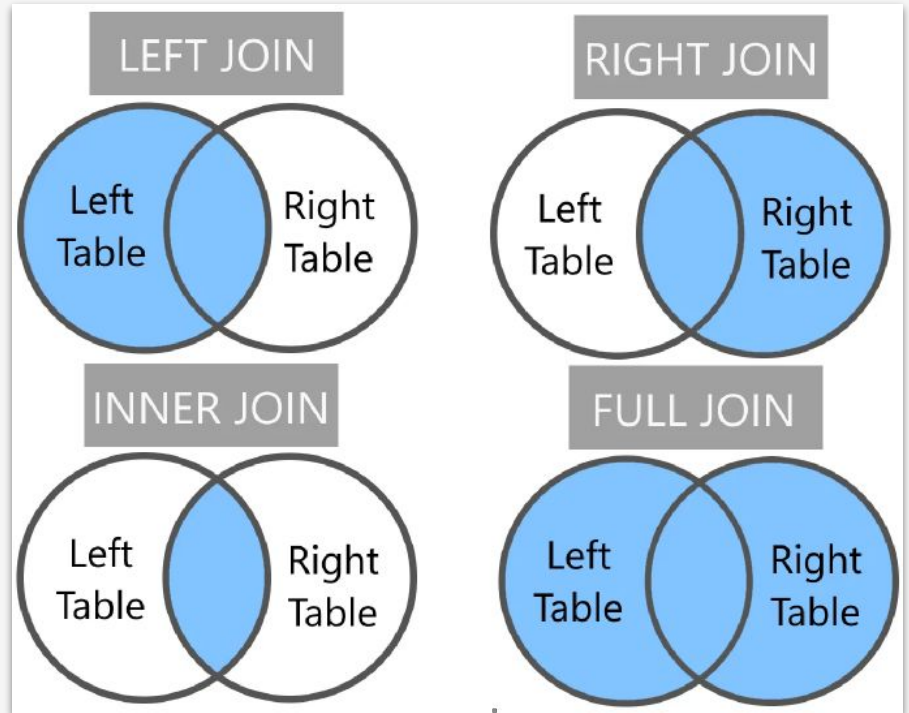
```
%%sql
SELECT type, SUM(cost), MIN(cost), MAX(name)
FROM Dish
GROUP BY type;
```

type	sum("cost")	min("cost")	max("name")
dessert	5	5	ice cream
entree	30	7	taco
appetizer	12	4	potsticker

JOIN

Specify joins between tables as part of the FROM statement

```
SELECT *  
FROM table1 (INNER) JOIN table2  
ON table1.key = table2.key
```



s

id	name
0	Apricot
1	Boots
2	Cally
4	Eugene

t

id	breed
1	persian
2	ragdoll
4	bengal
5	persian

```
SELECT *
FROM s
  INNER JOIN t
    ON s.id = t.id
```



s.id	name	t.id	breed
1	Boots	1	persian
2	Cally	2	ragdoll
4	Eugene	4	bengal

s

id	name
0	Apricot
1	Boots
2	Cally
4	Eugene

t

id	breed
1	persian
2	ragdoll
4	bengal
5	persian

```
SELECT *
FROM s
  LEFT OUTER JOIN t
    ON s.id = t.id;
```



s.id	name	t.id	breed
0	Apricot	–	–
1	Boots	1	persian
2	Cally	2	ragdoll
4	Eugene	4	bengal

s

id	name
0	Apricot
1	Boots
2	Cally
4	Eugene

t

id	breed
1	persian
2	ragdoll
4	bengal
5	persian

```
SELECT *
FROM s
  FULL OUTER JOIN t
    ON s.id = t.id;
```



s.id	name	t.id	breed
0	Apricot	–	–
1	Boots	1	persian
2	Cally	2	ragdoll
4	Eugene	4	bengal
–	–	5	persian

SQL Practice

Marshall Mathers' Data 100 study group is holding a mid-semester social on Memorial Glade, and he is tasked to get bubble tea (i.e., boba). Since it's a Data 100 event, he surveys the students and staff for some information about the boba shops they've been to in Berkeley, and compiles the data into a table named `BobaReview`. Each row is a person's review of a boba shop; the first few rows are shown to the right.

reviewer	role	shop_name	rating	price
Kelly	staff	Boba_Ninja	2	5
Andrew	staff	Asha	4.5	5
Parth	student	Asha	5	5
Kunal	staff	One_Plus	4.5	4.5

- (b) [4 Pts] Marshall now wants to find the **5 top-rated boba shops** in Berkeley. Help Marshall by completing the below SQL query, which selects the 5 boba shops from `BobaReview` with the highest average rating. Your query result should have three columns labeled `shop_name`, `avg_rating`, and `avg_price`, which are the shop name, average rating, and average price paid for boba, respectively. Rows should be sorted by the average rating, with the highest rated boba shop as the first row. The first few rows of the query result are shown below:

shop_name	avg_rating	avg_price
Asha	4.75	5.0
One_Plus	4.5	4.5

```
SELECT ____ (i) ____ FROM BobaReview ____ (ii) ____;
```

Sp22 MT2 Q1

Regular Ice, SQLite Sugar

- (b) [4 Pts] Marshall now wants to find the **5 top-rated boba shops** in Berkeley. Help Marshall by completing the below SQL query, which selects the 5 boba shops from BobaReview with the highest average rating. Your query result should have three columns labeled `shop_name`, `avg_rating`, and `avg_price`, which are the shop name, average rating, and average price paid for boba, respectively. Rows should be sorted by the average rating, with the highest rated boba shop as the first row. The first few rows of the query result are shown below:

shop_name	avg_rating	avg_price
Asha	4.75	5.0
One_Plus	4.5	4.5

```
SELECT ____ (i) ____ FROM BobaReview ____ (ii) ____;
```

- (i) What goes in the blank denoted by (i) ?

Solution:

```
shop_name,  
AVG(rating) AS avg_rating,  
AVG(price) AS avg_price
```

- (ii) What goes in the blank denoted by (ii) ? Use as few/many lines as you need.

Solution:

```
GROUP BY shop_name ORDER BY avg_rating DESC LIMIT 5;
```

Data Cleaning & EDA

Key Data Properties

1. Structure
2. Granularity
3. Temporality
4. Faithfulness

Structure

Structure: File formats (CSV, TSV, JSON) and variable types

There are two variable types: **quantitative** (numerical) and **qualitative** (categorical: ordinal/nominal).

- Ordinal: categories with ordered levels, but diff between levels has no quantifiable meaning
 - Ex: levels of education, income bracket, Yelp rating
- Nominal: categories with no specified order
 - Ex: political affiliation, student ID
- There are five possible stars on RateMyProfessor. What is the type of variable indicating number of stars for a particular professor? → *Qualitative ordinal*

Granularity

Granularity: What each row represents in the DataFrame.

- You can also think of it as the level of detail included in the data.
- What is the granularity of `df`? → *Each row represents an instance of iris flower*

```
df = sns.load_dataset('iris')  
df
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa

Temporality

Temporality:

- Periodicity over which the data was collected
- When the data was most recently collected or updated.

Faithfulness

Faithfulness: Ensuring our data accurately captures the “real world”

→ How do we deal with messy data?

→ How do we deal with missing data?

Three strategies:

1. Keep as NaN
2. Drop rows with missing data
 - Can introduce additional *bias*! Maybe there's a reason certain data is missing?
3. Impute missing values
 - Mean/median
 - Random non-NaN value
 - Regression

EDA Practice

Fall 2023 Midterm

1 Pound It, Noggin [18 Pts]

The YouTube channel Dude Perfect is known for its competition videos. In these multi-round competitions, members either compete individually or in teams of 2. Data collected from these videos is stored in a `DataFrame` called `winners`. Assume that each video only had one winning person/team. The first few rows (as well as descriptions of each column) are provided below.

- `Title`: Title of the video.
- `Rounds`: The number of rounds of competition that took place in the video.
- `Duration`: Duration of the video in minutes.
- `Views`: Millions of views the video received.
- `Name`: Name of the person or team who won.
- `Team Contest`: Binary column indicating 1 if it was a team competition, 0 if it was an individual competition.

	Title	Rounds	Duration	Views	Name	Team Contest
0	Angry Birds in Real Life	4	10.971	5.4	Garrett	0
1	The Survivor Games	6	21.422	7.5	Team Bluegrass	1
2	\$50,000 Crystal Treasure Hunt	1	13.974	6.6	Cory	0
3	World's Strongest Dude	4	13.853	16.0	Ty	0
4	Ultimate Mini Games Battle 3	13	12.522	16.0	Ty	0
5	Deep Sea Fishing Battle 2	1	13.658	8.3	Nacho Boat	1

	Title	Rounds	Duration	Views	Name	Team	Contest
0	Angry Birds in Real Life	4	10.971	5.4	Garrett		0
1	The Survivor Games	6	21.422	7.5	Team Bluegrass		1
2	\$50,000 Crystal Treasure Hunt	1	13.974	6.6	Cory		0
3	World's Strongest Dude	4	13.853	16.0	Ty		0
4	Ultimate Mini Games Battle 3	13	12.522	16.0	Ty		0
5	Deep Sea Fishing Battle 2	1	13.658	8.3	Nacho Boat		1

(a) [1 Pt] What is the **granularity** of winners? Respond in one sentence.

	Title	Rounds	Duration	Views	Name	Team	Contest
0	Angry Birds in Real Life	4	10.971	5.4	Garrett		0
1	The Survivor Games	6	21.422	7.5	Team Bluegrass		1
2	\$50,000 Crystal Treasure Hunt	1	13.974	6.6	Cory		0
3	World's Strongest Dude	4	13.853	16.0	Ty		0
4	Ultimate Mini Games Battle 3	13	12.522	16.0	Ty		0
5	Deep Sea Fishing Battle 2	1	13.658	8.3	Nacho Boat		1

(a) [1 Pt] What is the **granularity** of winners? Respond in one sentence.

Solution: Each row represents one YouTube video.

	Title	Rounds	Duration	Views	Name	Team Contest
0	Angry Birds in Real Life	4	10.971	5.4	Garrett	0
1	The Survivor Games	6	21.422	7.5	Team Bluegrass	1
2	\$50,000 Crystal Treasure Hunt	1	13.974	6.6	Cory	0
3	World's Strongest Dude	4	13.853	16.0	Ty	0
4	Ultimate Mini Games Battle 3	13	12.522	16.0	Ty	0
5	Deep Sea Fishing Battle 2	1	13.658	8.3	Nacho Boat	1

(b) [2 Pts] Which **variable type** best describes each of the following columns of winners?

QUANTITATIVE

Qualitative
Ordinal

Qualitative
Nominal

- | | | | | |
|---------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| (i) "Rounds" | ☐ | ☐ | ☐ | ☐ |
| (ii) "Duration" | ☐ | ☐ | ☐ | ☐ |
| (iii) "Name" | ☐ | ☐ | ☐ | ☐ |
| (iv) "Team Contest" | ☐ | ☐ | ☐ | ☐ |

(b) [2 Pts] Which **variable type** best describes each of the following columns of winners?

QUANTITATIVE

Qualitative
Ordinal

Qualitative
Nominal

(i) "Rounds"



(ii) "Duration"



(iii) "Name"



(iv) "Team Contest"



2 pan-DONS [8 Pts]

Welcome back to Day 100 of Guessing Brandon's lunch! Every day, members of the course staff take one guess each for what Brandon is going to eat for lunch. These predictions are stored in `guesses`, the first five rows of which are given to you below.

	Date	Name	Guess	Correct
0	10/18/2023	Matthew	Taco Sinaloa	0
1	10/18/2023	Yuerou	Toss Noodle	1
2	10/18/2023	Manas	Thai Basil	0
3	10/18/2023	Pragnay	Mezzo	0
4	10/17/2023	Manas	Bongo Burger	-99

Each day, Brandon goes through and manually fills in the `Correct` column with a value of 1 if the `Guess` was correct and 0 if the `Guess` was incorrect. However, every 3-4 days, he forgets what he had for lunch, and gives every guesser on that day a value of `-99` to represent a missing value (assume these values make up around 30% of the `Guess` column).

(a) [1 Pt] Which of the following is the **BEST** way we can deal with missing values (`-99`)?

- ☐ Dropping all rows with `-99` values
- ☐ Imputing with 0
- ☐ Imputing with 0.5
- ☐ Imputing with the average of that person's `Correct` column

(a) [1 Pt] Which of the following is the **BEST** way we can deal with missing values (-99)?

- ☐ Dropping all rows with -99 values
- ☐ Imputing with 0
- ☐ Imputing with 0.5
- ☒ **Imputing with the average of that person's Correct column**

Solution: In this scenario, we know that the person submitted a guess, but we don't know whether or not they were right. The best we can do is to go off the prior proportion of correct guesses that person had, which is the same as getting the mean of their previous Correct column.

Regex

What is Regex?

- A way of working with textual data!
 - Tying it in with EDA: we need some way way to clean up /process string data
- Made up of **literal characters** + **special characters**
- Literal Characters: alphanumerics (A-Z), numbers (0-9), etc.
- Special Characters: *, +, (), [], \, etc.

Strings + Series of Strings

Before diving into regex expressions, it's important to clarify how polars treats strings.

String --> String

```
s.lower()  
s.split()  
len(s)
```



Python assumes we are
working w 1 string at a time

Series of Strings --> Series of Strings

Pandas .str methods are vectorized

```
df['col1'].str.lower()  
df['col1'].str.split()  
df['col1'].str.lower().str.split()
```



.str methods are stackable!

series.str.string method

Regex Operations

Concatenation	Look for consecutive characters	r'AABBAB'	'AABBAB' ONLY
 → OR	Match one pattern or another	AA BBAB	'AA' or 'BBAB' ONLY
* → Zero or more	Match zero or more of a pattern	AB*A	Match: AA, ABBBBBA Won't Match: AB, ABABAB
() → Group	Used to group operations together and apply one regex op to multiple characters	A(A B)AAB	AAAAB, ABAAB
	Meaning	Pattern	Match

Regex Operations

.	Look for any character other than \n (newline)	.U.U.U.	UUU, CUMULUS
+	One or more character	AB+	AB, ABB, AB BB
{x}	Repeat exactly x times	AB{2}	ABB ONLY
{x, y}	Repeat between x + y times	AB{0,2}	A, AB, ABB
?	Zero or one times	AB?	A, AB
	Meaning	Pattern	Match

Regex Operations

[]	Defines an alphabet aka character class	[A-Za-z] [a-z0-9] [aeiou]	all lower case + capital letters, ex: AzaleaQ all lowercase letters & digits, ex: 2a34b all vowels, ex: aaaaiee
[^]	Negates a character class aka matches all characters EXCEPT the following character	[^A-Z]	any character that is NOT an uppercase letter, ex: 2z34b
\	Read next char literally	a\+b	a+b ('+' is no longer an operation)
^	Match beginning of a string	^abc	abc123, NOT 123abc
\$	Match end of a string	abc\$	123abc, NOT abc123
	Meaning	Pattern	Match

How to fix this?

Greediness

Regex is GREEDY, meaning it looks for the longest possible match in a string.

String =
'aaaaabaac'
Pattern = a+ba*
Match =
aaaaabaa

String= "This is an <div>example</div>
of greediness <div>in</div> regular
expressions."
Pattern = <div>.*</div>
Match = <div>example</div> of
greediness <div>in</div>

Greediness

We can fix greediness with the ? tags

```
<div>.*?</div>
```

- 1 What we are telling Regex:
- 2 Look for the exact string '<div>'
- 3 Grab every character except \n
Until the FIRST instance of '</div>'

*? , +?

The ? tags multipliers as
non-greedy

Read more [HERE](#)

"This is an <div>example</div> of greediness <div>in</div> regular
expressions."

Regex Practice

Fa24 MT Q4C

Pur+egex!

(c) [1 Pt] Given the provided variables:

```
case = "Can you let the mouse go, Jordan?"  
pattern = r"\w?$"
```

What is the output of the following function call?

```
re.search(pattern, case)[0]
```

Select the correct option from below.

- ☐ "Jordan"
- ☐ "n"
- ☐ "n?"
- ☐ ""

Fa24 MT Q4C

Pur+egex!

[SOLUTION]

(c) [1 Pt] Given the provided variables:

```
case = "Can you let the mouse go, Jordan?"  
pattern = r"\w?$"
```

What is the output of the following function call?

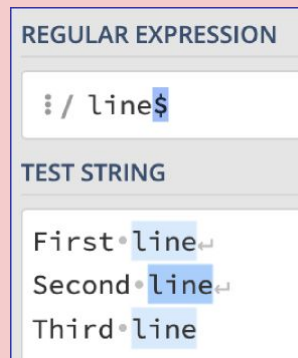
```
re.search(pattern, case)[0]
```

Select the correct option from below.

- ☐ "Jordan"
- ☐ "n"
- ☐ "n?"

☐ "" the matched end of line must terminate with an alphanumeric character, or no character at all

In the example `r"line$"`, we
match end of each line with `$`



Sp25 MT Q2A

Hint: `re.findall` extracts only the text inside of the **group**

- (a) [3 Pts] Sabrina found metadata in each staff introduction to process as strings.

Suppose Sabrina has already created a pattern and runs the code below. The character "_" represents a single space, and you may use it in your response.

```
metadata = "Name: Oski; Age: 159; Courses taken: Data8,
Data100, CS61A, Stat134, Data88s; Phone Number:
555-100-5555; Likes: [Bears, Strawberries, Data Science]"
```

```
pattern = r"\w+:\s([A-Za-z\s]+)"
```

```
re.findall(pattern, metadata)
```

List matches in the order returned by `re.findall(pattern, metadata)`, with the first match next to Match 1.

Note: You may have less than 6 total matches. For any unmatched slots, **write** No Match instead.

Sp25 MT Q2A

[SOLUTION]

```
metadata = "Name: Oski; Age: 159; Courses taken: Data8,  
Data100, CS61A, Stat134, Data88s; Phone Number:  
555-100-5555; Likes: [Bears, Strawberries, Data Science]"
```

```
pattern = r"\w+:\s([A-Za-z\s]+)"
```

```
re.findall(pattern, metadata)
```

We match until encountering a letter or space

Solution:

Match 1: '159;'

Match 2: '555-100-5555;'

Match 3: '['

Match 4: 'No Match'

Match 5: 'No Match'

Match 6: 'No Match'

Fa25 Final Q1 iv

- ☐ True

☐ False

r"([FallSpring]_\d+) "
- ☐ True

☐ False

r"(Fall|Spring)\s\d{4}"
- ☐ True

☐ False

r"during_the_(\w{,6}_\d{4})_semester"
- ☐ True

☐ False

r"(Fall_\d*|Spring_\d*)"

[2 Pts] Jake creates a new Semester column that contains the semester mentioned in the Team Description column. Jake writes the following skeleton code:

```
basketball['Semester'] =  
    basketball['Team Description'].str.extract(__ (iv) __)
```

Which of the following regex patterns correctly fills the blank (iv) to create the Semester column? For example, your chosen regex pattern should extract the text "Fall_2025" from the example below:

"D(100)_Hoopers_played_during_the_Fall_2025_semester_on_Monday"

Note: The special character _ refers to a single space.

Hint: str.extract extracts only the text inside of the **group**

Assumptions: All entries in the Team Description column follow the format below. No team name contains the words Spring or Fall. IM basketball does not happen during the summer semester.

{Team_Name}_played_during_the_{Fall/Spring}_{Year}_semester_on_{Day}

Fa25 Final Q1 iv

[SOLUTION]

Assumptions: All entries in the Team Description column follow the format below. No team name contains the words Spring or Fall. IM basketball does not happen during the summer semester.

{Team_Name}_played_during_the_{Fall/Spring}_{Year}_semester_on_{Day}

- ☐ True ☒ **False** `r"([FallSpring]_\d+)"` >[] indicates character class. this matches "g 2025"
- ☐ True ☒ **False** `r"(Fall|Spring)\s\d{4}"` Only the season name will be extracted
- ☒ **True** ☐ False `r"during_the_(\w{,6}_\d{4})_semester"`
- ☒ **True** ☐ False `r"(Fall_\d*|Spring_\d*)"`

Visualizations

Goals of Visualization

- **Before/during analysis: Broaden understanding of data**
 - Summarize trends visually is key in EDA
 - Lightweight, iterative and flexible process to see relationships between variables
- **After analysis: Communicate results/conclusions**
 - Highly editorial, selective, and fine-tuned to achieve a communications goal
 - be thoughtful and careful about its clarity, accessibility, and necessary context.

Matplotlib and Seaborn

- Both libraries specifically built for plotting and visualizing data

```
import matplotlib.pyplot as plt # matplotlib is typically given the alias plt

continent = wb['Continent'].value_counts()
plt.bar(continent.index, continent)
plt.xlabel('Continent')
plt.ylabel('Count');
```

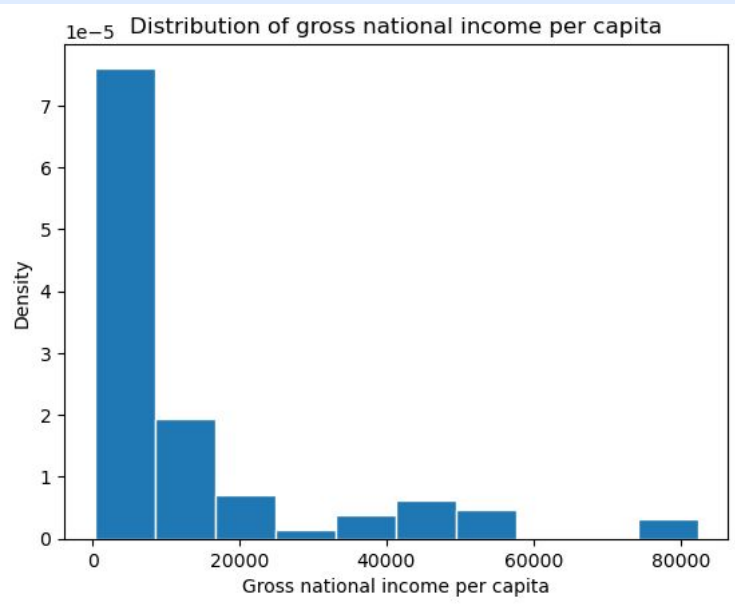
- But need to define labels explicitly...
- **Seaborn** is built on **Matplotlib**! (Matplotlib happening under the hood)
 - Easier-to-use interface for working with DataFrames and creating certain types of plots

```
import seaborn as sns # seaborn is typically given the alias sns
sns.countplot(data = wb, x = 'Continent', hue='Continent');
```

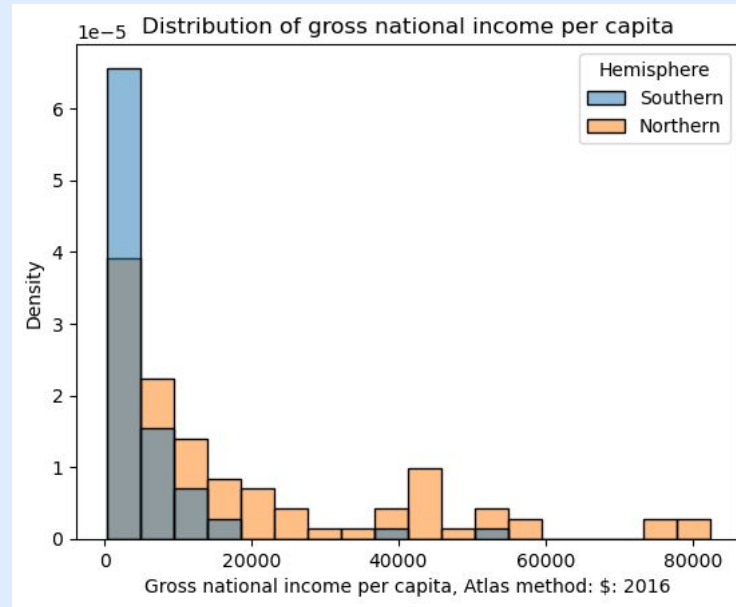
Distribution of one variable

Variable type(s)	Visualization	What it shows
Quantitative	Histogram	Distribution (shape, center, spread, skew)
Quantitative	Boxplot	Summary statistics (median, IQR, outliers)
Quantitative	Violin plot	Summary statistics (median, IQR, outliers), shows modality
Qualitative	Bar chart	Counts/frequency of each category

Distribution of one variable - Histograms



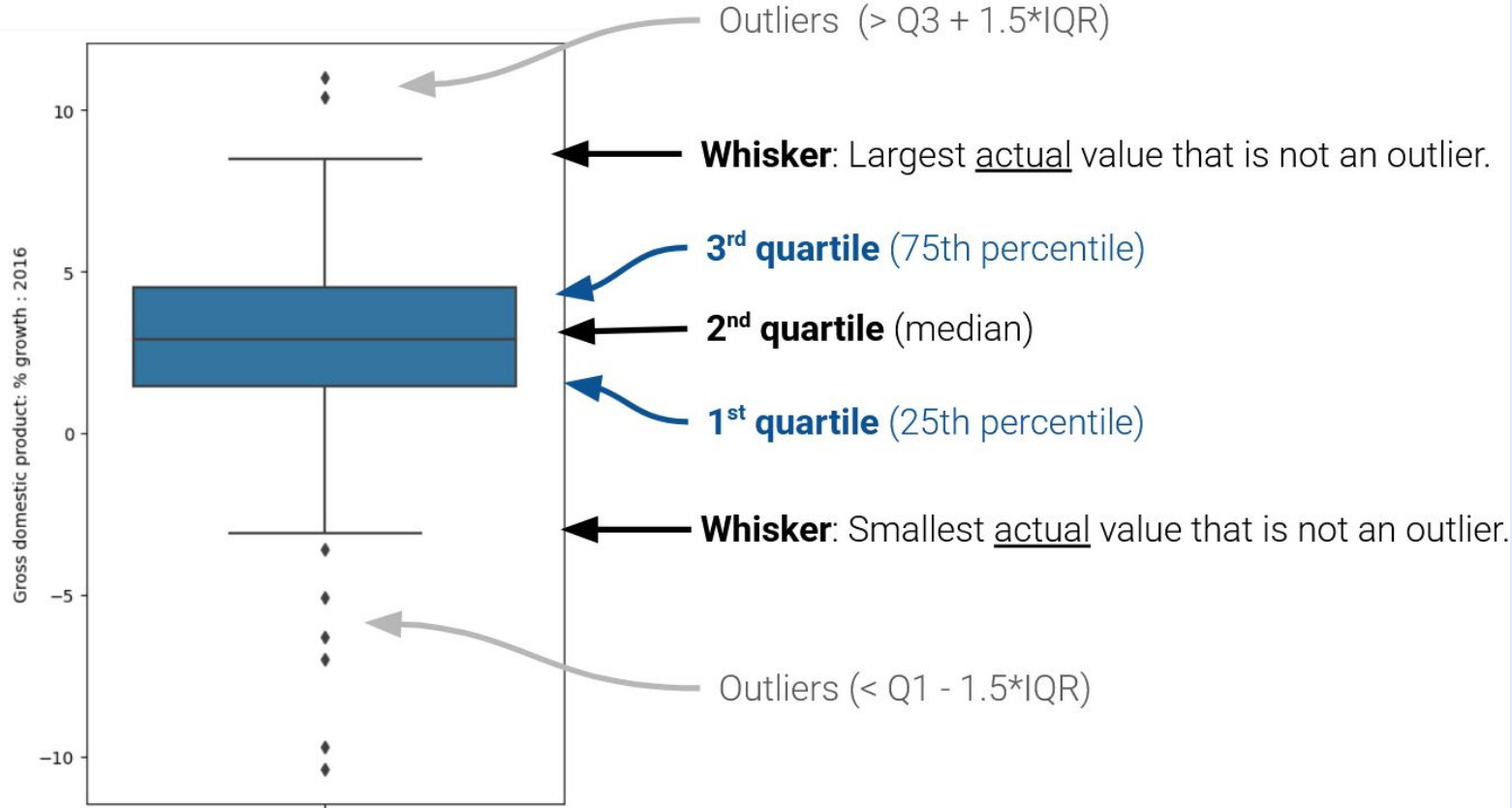
Density histogram:
area = proportion of data



Overlaid histograms

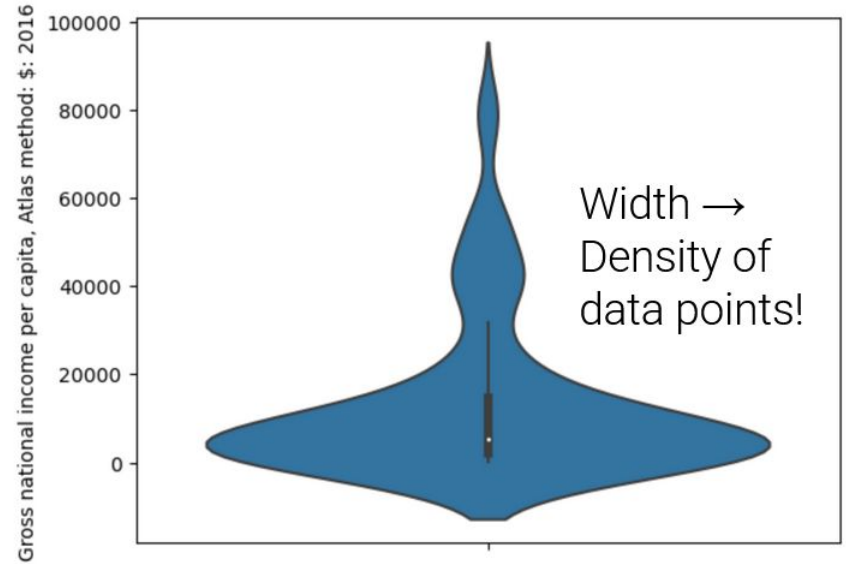
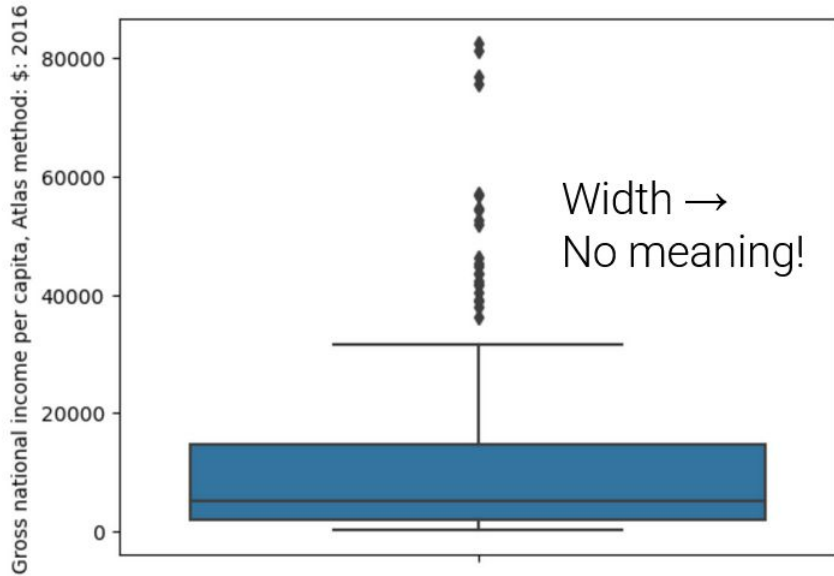
Right or left skew?

Distribution of one variable - Box plots



Right or left
skew?

Distribution of one variable - Box plots vs. violin plots

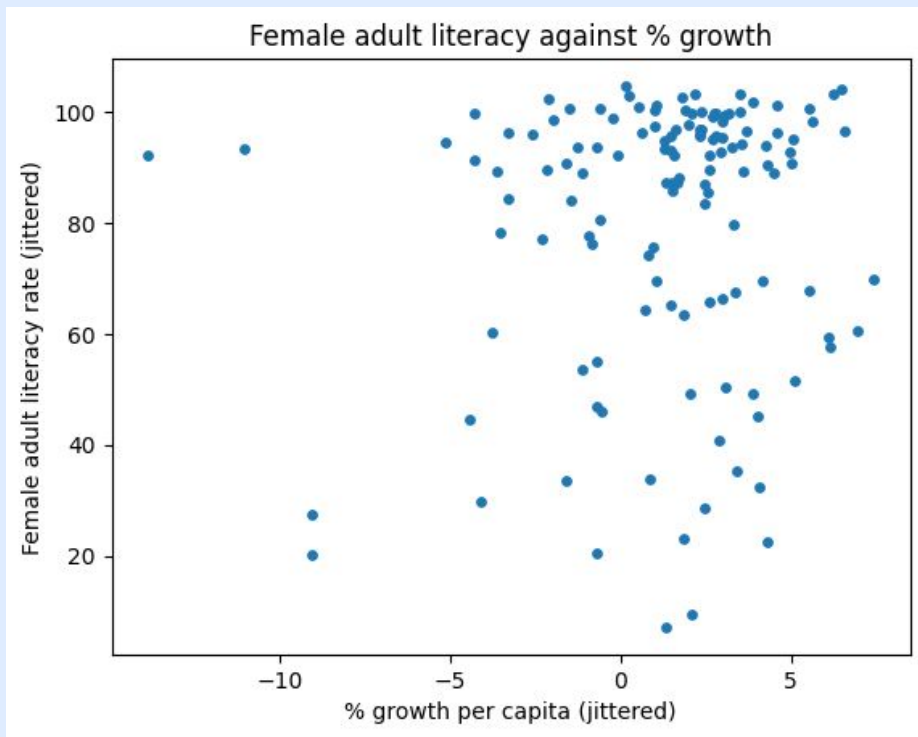


Violin plot shows **shape** and **modality** of distribution

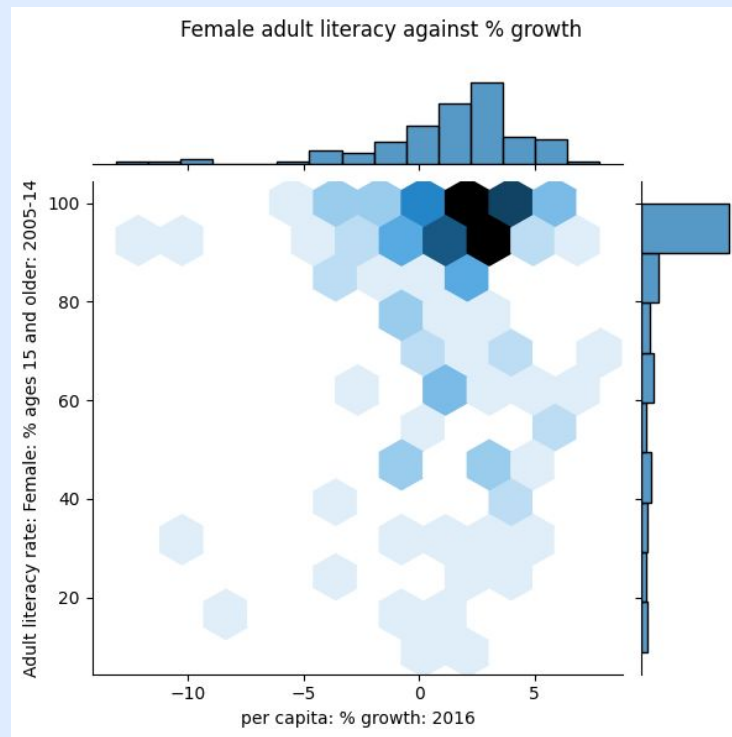
Two variables

Variable type(s)	Visualization	What it shows
Quant + Quant	Scatter plot	Relationship (correlation, trends, outliers)
Quant + Quant	Hex plot	Relationship (correlation, trends, outliers); reduces overplotting
Quant + Quant	Line plot	Temporal trends over time
Quant + Qual	Boxplot per category	Distribution of values per category
Quant + Qual	Violin plot per category	Shape and distribution of values per category
Qual + Qual	Stacked bar chart	Part-to-whole within categories

Two variables: scatter plot vs. hex plot



Scatter plot: show individual datapoints

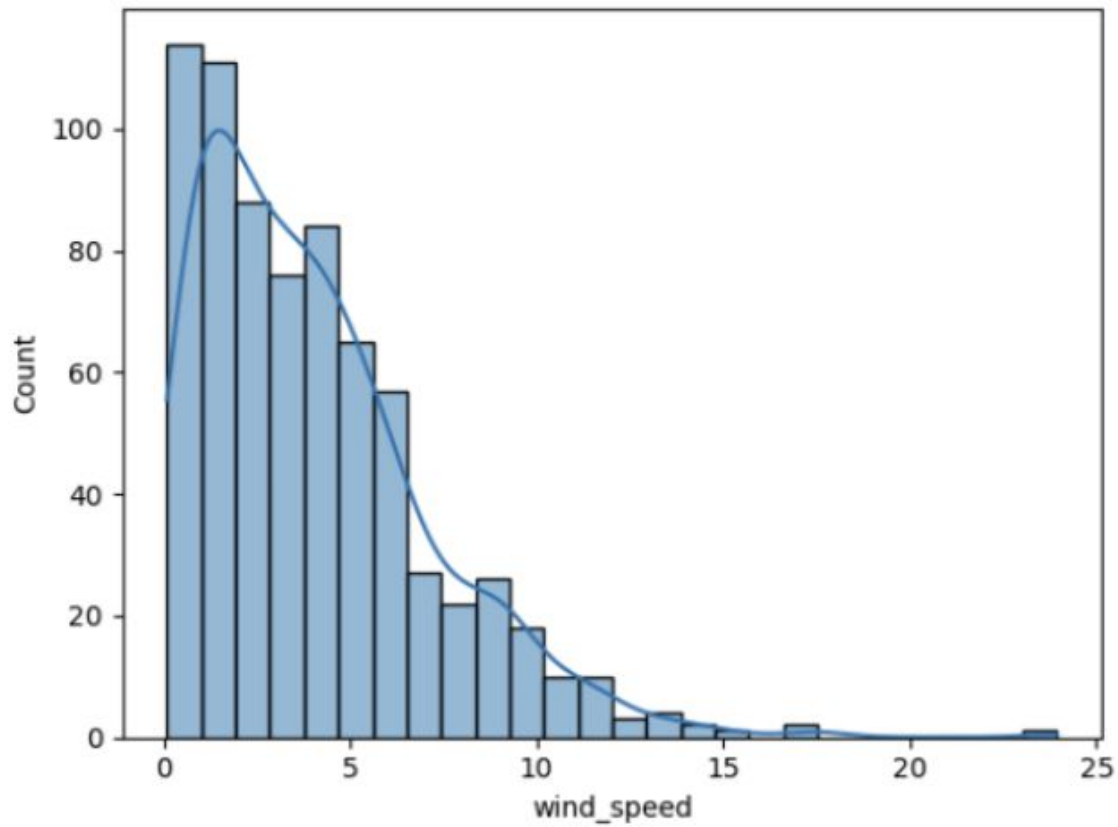


Hex plot: show trends, reduce overplotting

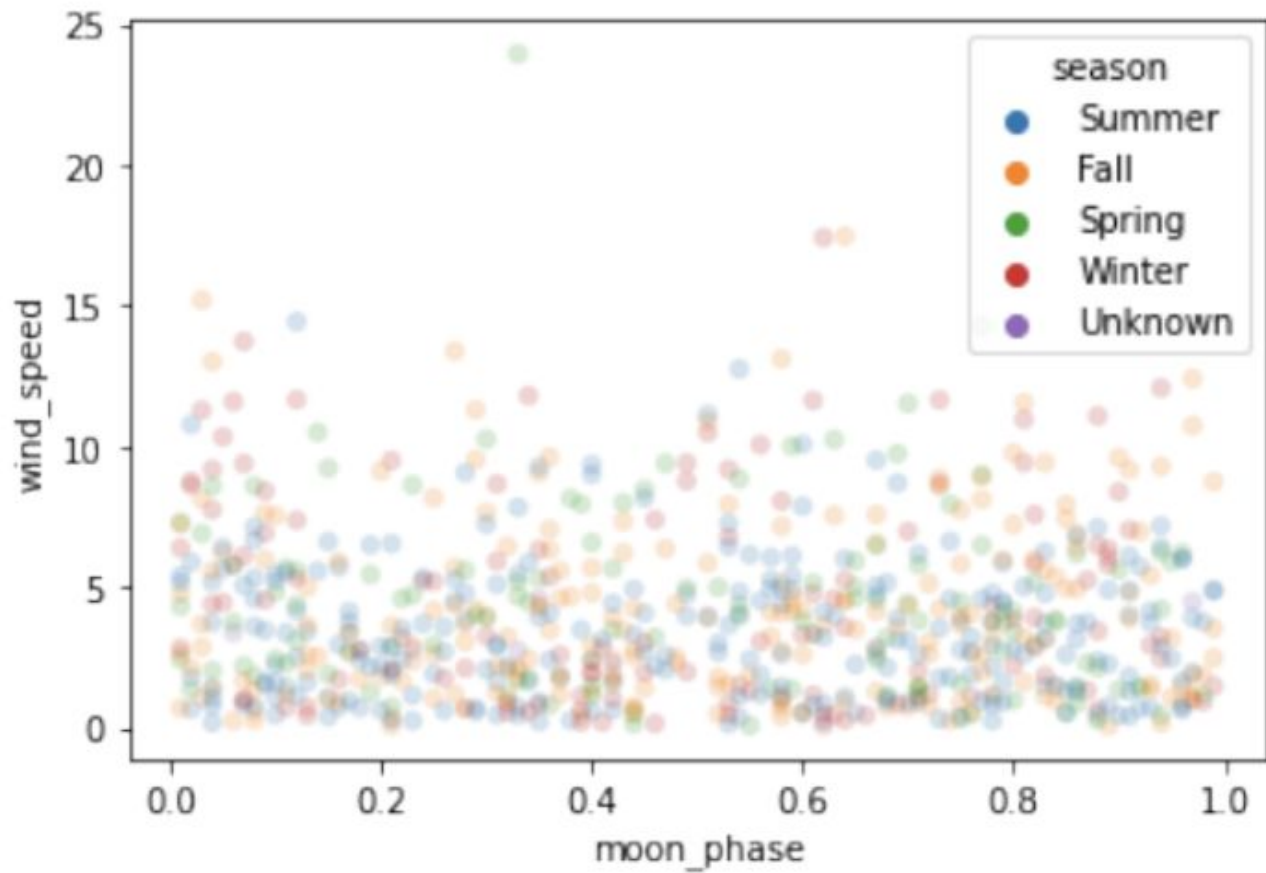
Three and more variables

Variable type(s)	Best visualization	What it shows
All Quant	Heatmap e.g. correlation matrix	Strength and direction of variable correlations
Mixed Quant + Qual	Color or shape in scatter plot	Adds third (or more) variables via encoding

Visualizations Practice



How many channels of information? How many dimensions?



How many channels of information? How many variables / features?

Spring 2023 Midterm Q4

For this question, assume that the `seaborn` library has been imported as `sns` and the `matplotlib.pyplot` library has been imported as `plt`.

Ishani is a fantasy sports champion who wants to use her data science skills to get a head start next season. She records last season's players' team, position, number of games played, and average number of fantasy points per game, storing the results in a `DataFrame` called `players`. The first 5 rows are given to you below.

	name	team	position	games_played	points_per_game
0	CeeDee Lamb	DAL	WR	17	23.7
1	Josh Allen	BUF	QB	17	23.1
2	Christian McCaffrey	SF	RB	16	24.5
3	Tyreek Hill	MIA	WR	16	23.5
4	Jalen Hurts	PHI	QB	17	21.0

(a) [2 Pts] Which of the following visualization types are appropriate to visualize the distribution of `games_played`? **Select all that apply.**

☐ A. Histogram

☐ C. Violinplot

☐ B. Hexplot

☐ D. Scatterplot

(a) [2 Pts] Which of the following visualization types are appropriate to visualize the distribution of games_played? **Select all that apply.**

☐ **A. Histogram**

☐ **C. Violinplot**

☐ B. Hexplot

☐ D. Scatterplot

Solution:

Options A and C are both common choices to display the distribution of one quantitative column.

Options B and D both display the relationship between two quantitative variables.

(b) [2 Pts] Ishani creates a DataFrame called `output` using the following code:

```
total = players["games_played"] * players["points_per_game"]
players["total"] = total
output = players.groupby("team")["total"].mean().reset_index()
```

Which of the following lines of code are best suited to visualize the relationship between both columns of `output`? **Select all that apply.**

- ☐ A. `sns.violinplot(data = output, x = "total");`
- ☐ B. `sns.boxplot(data = output, x = "team", y = "total")`
- ☐ C. `plt.bar(x = output["team"], height = output["total"])`
- ☐ D. `sns.countplot(data = output, x = "team")`

(b) [2 Pts] Ishani creates a DataFrame called output using the following code:

```
total = players["games_played"] * players["points_per_game"]
players["total"] = total
output = players.groupby("team")["total"].mean().reset_index()
```

Which of the following lines of code are best suited to visualize the relationship between both columns of output? **Select all that apply.**

- ☐ A. `sns.violinplot(data = output, x = "total");`
- ☐ B. `sns.boxplot(data = output, x = "team", y = "total")`
- ☒ C. `plt.bar(x = output["team"], height = output["total"])`
- ☐ D. `sns.countplot(data = output, x = "team")`

Solution:

output displays the average number of points accrued by players of each team. A good visualization will include both the team and their points.

As such, ~~Option B will create a boxplot (with one item) for each team,~~ while Option C will create a bar chart showing the average points per team.

Option A does not include any information about the teams, and Option D only shows how many times each team shows up in output.

(c) [2 Pts] Which of the following pieces of information can you **always** determine by looking at a box plot? **Select all that apply.**

☐ A. Quartiles

☐ C. Mean

☐ B. Existence of outliers

☐ D. Mode

(c) [2 Pts] Which of the following pieces of information can you **always** determine by looking at a box plot? **Select all that apply.**

☐ **A. Quartiles**

☐ C. Mean

☐ **B. Existence of outliers**

☐ D. Mode

Solution:

Option A is correct. The central box captures the first, second, and third quartiles.

Option B is correct. Values outside the whiskers are considered outliers and plotted as independent dots.

Option C is incorrect. Boxplots do not plot the mean.

Option D is incorrect. This is not something a boxplot can do.

Sasha Murray and Ved Barakam

Thank you for coming!

SP24 MT Q1C

Panda Express

- (c) [2 Pts] Milad determines that the main ingredient of each item is its last word (i.e., the main ingredient of "Orange Chicken" is "Chicken"). He wishes to store each main ingredient in a new column in orders called main, as seen below:

	name	item	size	veg	premium	price	main
0	Sean	Orange Chicken	large	0	0	11.4	Chicken
1	Brandon	Chow Mein	medium	1	0	8.7	Mein
2	Brandon	Eggplant Tofu	small	1	0	5.4	Tofu
3	Jessica	Honey Walnut Shrimp	medium	0	1	11.7	Shrimp
4	Sean	Broccoli Beef	small	0	0	5.4	Beef

Which of the following options will correctly fill in the blank to create the main column?
Select all that apply.

`orders["main"] = _____`

- ☐ A. `orders["item"].str.split().str[-1]`
- ☐ B. `orders["item"].str[-1]`
- ☐ C. `orders["item"].str.contains(r"\s*(\w+) ")`
- ☐ D. `orders["item"].str.extract(r"(\w+) $")`

SP24 MT Q1C

Panda Express

[SOLUTION]

Options A, D are correct.

Note that we always have to specify a capture group for `.str.extract()` argument.

Option B will return only last character, not word.

Option C will return a Series of booleans instead of words. The capture group here does not do anything.

- (c) [2 Pts] Milad determines that the main ingredient of each item is its last word (i.e., the main ingredient of "Orange Chicken" is "Chicken"). He wishes to store each main ingredient in a new column in orders called `main`, as seen below:

	name	item	size	veg	premium	price	main
0	Sean	Orange Chicken	large	0	0	11.4	Chicken
1	Brandon	Chow Mein	medium	1	0	8.7	Mein
2	Brandon	Eggplant Tofu	small	1	0	5.4	Tofu
3	Jessica	Honey Walnut Shrimp	medium	0	1	11.7	Shrimp
4	Sean	Broccoli Beef	small	0	0	5.4	Beef

Which of the following options will correctly fill in the blank to create the `main` column?
Select all that apply.

`orders["main"] = _____`

- ☐ A. `orders["item"].str.split().str[-1]`
- ☐ B. `orders["item"].str[-1]`
- ☐ C. `orders["item"].str.contains(r"\s*(\w+)")`
- ☐ D. `orders["item"].str.extract(r"(\w+)$")`

SP25 F Q1E

Ordering Chicken Nuggets for the Table

	chain	location	shape	weight
0	McDonald's	123 San Pablo Ave, 94702	oval	0.60
1	Wendy's	245 Telegraph Ave, 94704	square	0.58
2	Wendy's	88 Shattuck Square, 94709	circle	0.70
3	Chick-fil-A	300 Shattuck Ave, 94705	circle	0.70
4	Burger King	19 Ashby Blvd, 94703	crown	0.60

(e) [2 Pts] Fill in the blank to extract the **street name** from each location in the nuggets DataFrame. Store the resulting values in a new column called street.

```
nuggets["street"] = nuggets["location"].str.extract(r"_____")
```

Note 1: Each location is formatted as:

```
"StreetNumber_StreetName,_5DigitZIPCode"
```

The character "_" represents a single space, and you may use it in your response.

Note 2: StreetNumber contains only digits (i.e., no letters, _, punctuation, or special characters). StreetNumber can start with any digit, including 0. There will always be at least one digit in every StreetNumber.

Note 3: StreetName only contains uppercase and lowercase letters in the English alphabet and _ (i.e., no numbers, punctuation, or special characters). There will always be at least one letter in every StreetName.

Example:

- location: "123_San_Pablo_Ave,_94702"
- street: "San_Pablo_Ave"

Fill in the blank with the appropriate RegEx pattern:

SP25 F Q1E

Ordering Chicken Nuggets

for the Table

[SOLUTION]

	chain	location	shape	weight
0	McDonald's	123 San Pablo Ave, 94702	oval	0.60
1	Wendy's	245 Telegraph Ave, 94704	square	0.58
2	Wendy's	88 Shattuck Square, 94709	circle	0.70
3	Chick-fil-A	300 Shattuck Ave, 94705	circle	0.70
4	Burger King	19 Ashby Blvd, 94703	crown	0.60

Solution: Many possibilities, including:

1. `\d+\s([A-Za-z\s]+)`
2. `\d+\s([\w\s]+)`
3. `\d+\s(.\+?),`
4. `\d+\s(.+),\s\d{5}$`

(e) [2 Pts] Fill in the blank to extract the **street name** from each location in the `nuggets` DataFrame. Store the resulting values in a new column called `street`.

```
nuggets["street"] = nuggets["location"].str.extract(r"_____")
```

Note 1: Each location is formatted as:

`"StreetNumber_StreetName,_5DigitZIPCode"`

The character `"_"` represents a single space, and you may use it in your response.

Note 2: `StreetNumber` contains only digits (i.e., no letters, `_`, punctuation, or special characters). `StreetNumber` can start with any digit, including 0. There will always be at least one digit in every `StreetNumber`.

Note 3: `StreetName` only contains uppercase and lowercase letters in the English alphabet and `_` (i.e., no numbers, punctuation, or special characters). There will always be at least one letter in every `StreetName`.

Example:

- location: `"123_San_Pablo_Ave,_94702"`
- street: `"San_Pablo_Ave"`

Fill in the blank with the appropriate RegEx pattern:

```
r"\d+\s([A-Za-z]+),\s\d{5}"
```

(a) Aneesh decides to do the following RegEx exercises before processing feedback.

Fa24 MT Q4Ai,ii

Pur+egex!

- (i) [2 Pts] In the following blank, write a RegEx pattern that only matches words that contain the substring "cat" and does not contain any uppercase letters or space characters.

Fill in the blank with only the RegEx pattern, **do not make your solution a raw string**. A raw string is in the format of r"_____".

Match all of these below

✓cats
✓concatenate
✓10cats10

Match none of these below

✗Cat
✗CA Tacoma
✗CATcatCAT

- (ii) [1 Pts] Assume the correct answer to the previous question is stored in a raw string `pat`, and `cat_responses` is a Series of string objects. Write a line of pandas code that replaces any word that matches `pat` with "dog".

Fa24 MT Q4Ai,ii

Pur+egex!

[SOLUTION]

(a) Aneesh decides to do the following RegEx exercises before processing feedback.

- (i) [2 Pts] In the following blank, write a RegEx pattern that only matches words that contain the substring "cat" and does not contain any uppercase letters or space characters.

Fill in the blank with only the RegEx pattern, **do not make your solution a raw string**. A raw string is in the format of r"_____".

Match all of these below

✓cats
✓concatenate
✓10cats10

Match none of these below

✗Cat
✗CA Tacoma
✗CATcatCAT

```
[^A-Z\s]*cat[^A-Z\s]*
```

- (ii) [1 Pts] Assume the correct answer to the previous question is stored in a raw string `pat`, and `cat_responses` is a Series of string objects. Write a line of pandas code that replaces any word that matches `pat` with "dog".

```
cat_responses.str.replace(pat, "dog", regex=True)
```

Fa24 MT Q4B

Pur+egex!

(b) [2 Pts] Here is the provided table:

Match all of these below

✓total cats petted:60,102,305

✓my cats' names are:amy,baron,carol

✓the current time is 20:59

For each of the following RegEx patterns, determine if it is true that they fully match all cases listed in the “Match all of these below” column.

- ☐ True ☐ False `.*:\w+(,\w+)*`
- ☐ True ☐ False `.*:[A-Za-z\d]+(,[A-Za-z\d]+)*`
- ☐ True ☐ False `.*:\w+(,\w+)+`
- ☐ True ☐ False `\D*:\w+(,\w+)+`

Fa24 MT Q4B

Pur+egex!

[SOLUTION]

(b) [2 Pts] Here is the provided table:

Match all of these below

✓total cats petted:60,102,305

✓my cats' names are:amy,baron,carol

✓the current time is 20:59

For each of the following RegEx patterns, determine if it is true that they fully match all cases listed in the “Match all of these below” column.

- ☒ **True** ☐ False `. * : \w+ ( , \w+ ) *`
- ☒ **True** ☐ False `. * : [A-Za-z\d] + ( , [A-Za-z\d] + ) *`
- ☐ True ☒ **False** `. * : \w+ ( , \w+ ) +`
- ☐ True ☒ **False** `\D * : \w+ ( , \w+ ) +`

Last two options assume at least one comma followed by a 1 or more alphanumeric character(s)

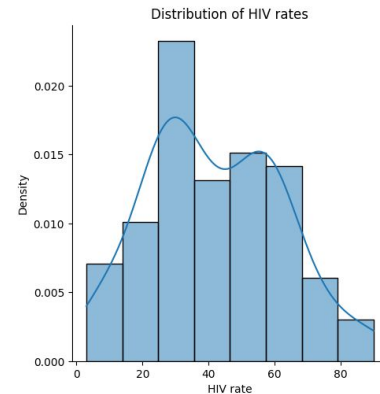
Kernel Density Estimation (KDE)

- Smooth, continuous function that approximates a curve
- Represent general trends in a distribution without focusing on the details
 - Useful for analyzing the broad structure of a dataset
- KDE attempts to **approximate the underlying probability distribution** from which our data was drawn; estimate the probability density function of a random variable
 - **Probability distribution:** Description of how likely to sample a particular value in data

```
import seaborn as sns
import matplotlib.pyplot as plt

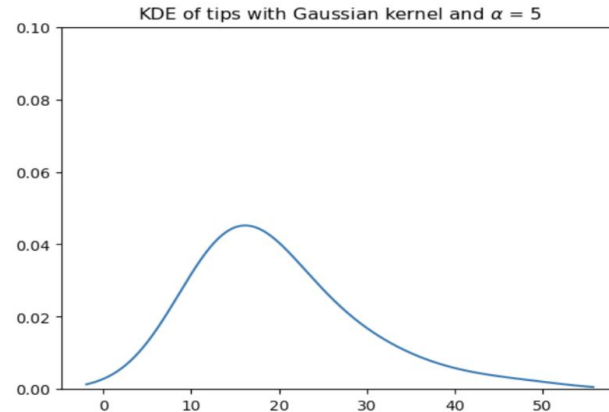
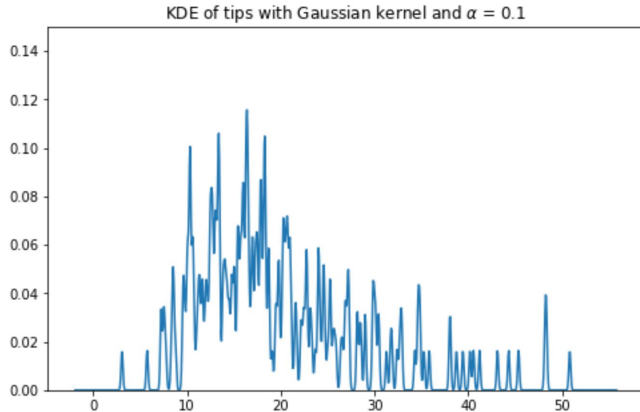
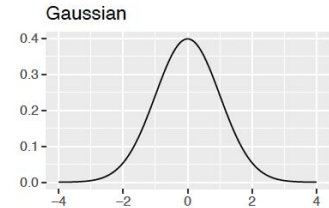
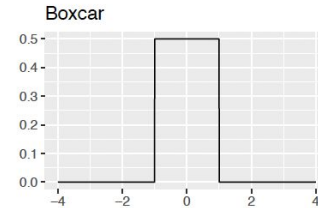
sns.displot(data = wb, x = 'HIV rate', \
            kde = True, stat = "density")

plt.title("Distribution of HIV rates");
```



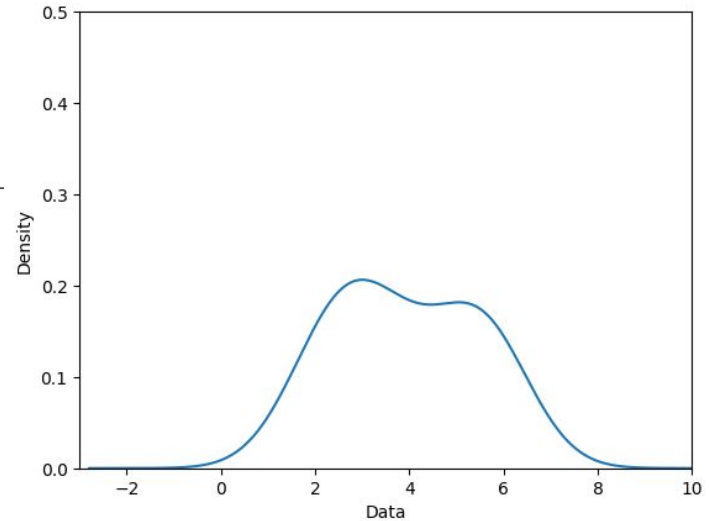
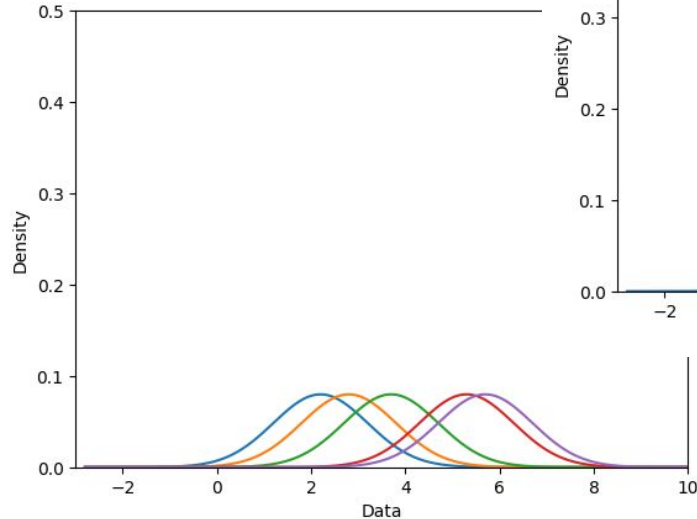
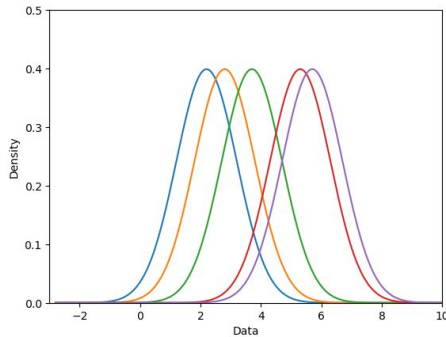
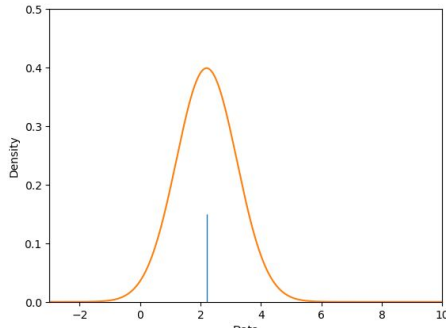
Kernel Density Estimation (KDE) - Definitions

- **Kernel:** Type of density curve; Mathematical function that attempts to capture randomness of each data point
- **Bandwidth:** Width of the kernel
 - Large $\alpha \rightarrow$ wide and short; Small $\alpha \rightarrow$ narrow and tall



Kernel Density Estimation (KDE) - Construction

1. Place a kernel at each datapoint
2. Normalize the kernels to have a total area of 1 (across all kernels)
3. Sum the normalized kernels



Kernel Density Estimation (KDE) - Mathematical Notation

$$f_{\alpha}(x) = \frac{1}{n} \sum_{i=1}^n K_{\alpha}(x, x_i)$$

(Note: In the original image, the '2' in f_{α} is labeled with a blue box containing '2', the '3' in $\sum_{i=1}^n$ is labeled with a blue box containing '3', and the '1' in K_{α} is labeled with a blue box containing '1'.)

1. $f_{\alpha}(x)$ is the end result. → **Height** of the final KDE curve at any x-value.
2. $K_{\alpha}(x, x_i)$ is the kernel centered on the observation x_i .
 - Each kernel individually has area 1.
 - x represents any number on the number line. It is the input to our function.
3. n is the number of observed datapoints that we have.
 - We multiply by $\frac{1}{n}$ so that the total area of the KDE is still 1.
4. Each $x_i \in \{x_1, x_2, \dots, x_n\}$ represents an observed datapoint.
 - These are what we use to create our KDE by summing multiple shifted kernels centered at these points.

- (d) [3 Pts] For the subparts of this question, select the best choice to fill in each blank of the below statement:

The first step of making a Kernel Density Estimate (KDE) curve is to place a kernel at ____A____. Next, you normalize each kernel so that ____B____ and sum the normalized kernels. One key value to set is the bandwidth parameter α , which determines the ____C____ of our KDE curve.

(i) Fill in Blank A:

- ☐ A. The minimum and maximum
- ☐ B. Every data point
- ☐ C. Some data points, depending on α
- ☐ D. None of the above

(ii) Fill in Blank B:

- ☐ A. Each kernel has an area of α
- ☐ B. Each kernel has an area of 1
- ☐ C. The total area under the KDE curve is 1
- ☐ D. None of the above

(iii) Fill in Blank C:

- ☐ A. Height
- ☐ B. Smoothness
- ☐ C. Area
- ☐ D. None of the above

- (d) [3 Pts] For the subparts of this question, select the best choice to fill in each blank of the below statement:

The first step of making a Kernel Density Estimate (KDE) curve is to place a kernel at ____A____. Next, you normalize each kernel so that ____B____ and sum the normalized kernels. One key value to set is the bandwidth parameter α , which determines the ____C____ of our KDE curve.

(i) Fill in Blank A:

- ☐ A. The minimum and maximum
- ☒ **B. Every data point**
- ☐ C. Some data points, depending on α
- ☐ D. None of the above

(ii) Fill in Blank B:

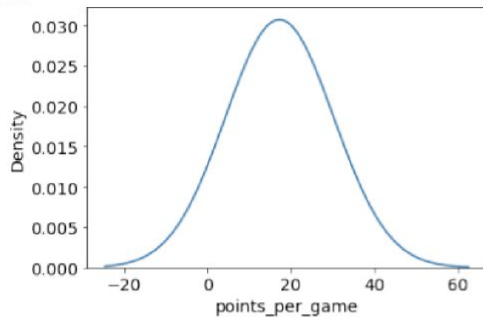
- ☐ A. Each kernel has an area of α
- ☐ B. Each kernel has an area of 1
- ☒ **C. The total area under the KDE curve is 1**
- ☐ D. None of the above

(iii) Fill in Blank C:

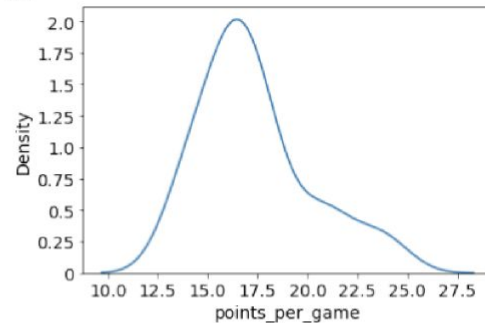
- ☐ A. Height
- ☒ **B. Smoothness**
- ☐ C. Area
- ☐ D. None of the above

(e) [1 Pt] Ishani generates the following KDE curves from the `points_per_game` column:

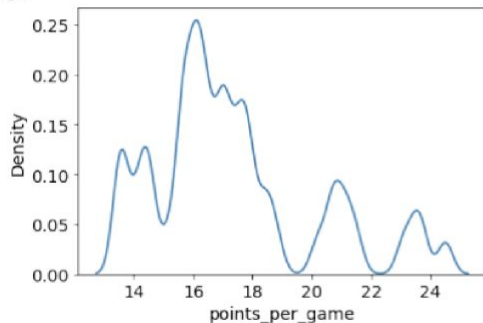
A.



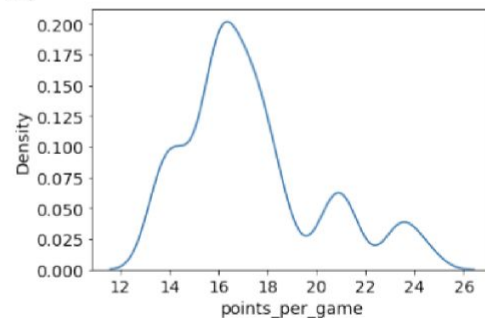
B.



C.



D.



Which of the above plots is **NOT** a valid KDE curve?

☐ A

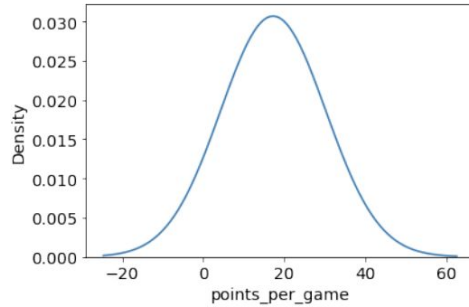
☐ B

☐ C

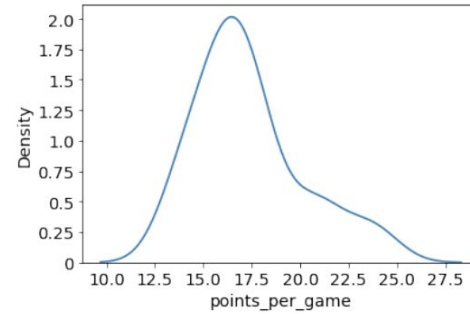
☐ D

(e) [1 Pt] Ishani generates the following KDE curves from the `points_per_game` column:

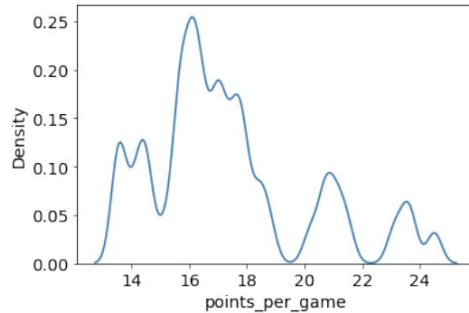
A.



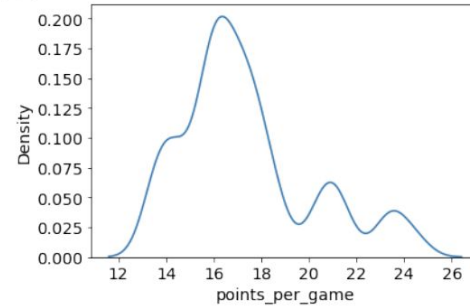
B.



C.



D.



Which of the above plots is **NOT** a valid KDE curve?

☐ A

☒ B

☐ C

☐ D

Solution: The total area under a KDE curve has to be 1, which is violated by Option B.

Variable Types Should Inform Plot Choice

